



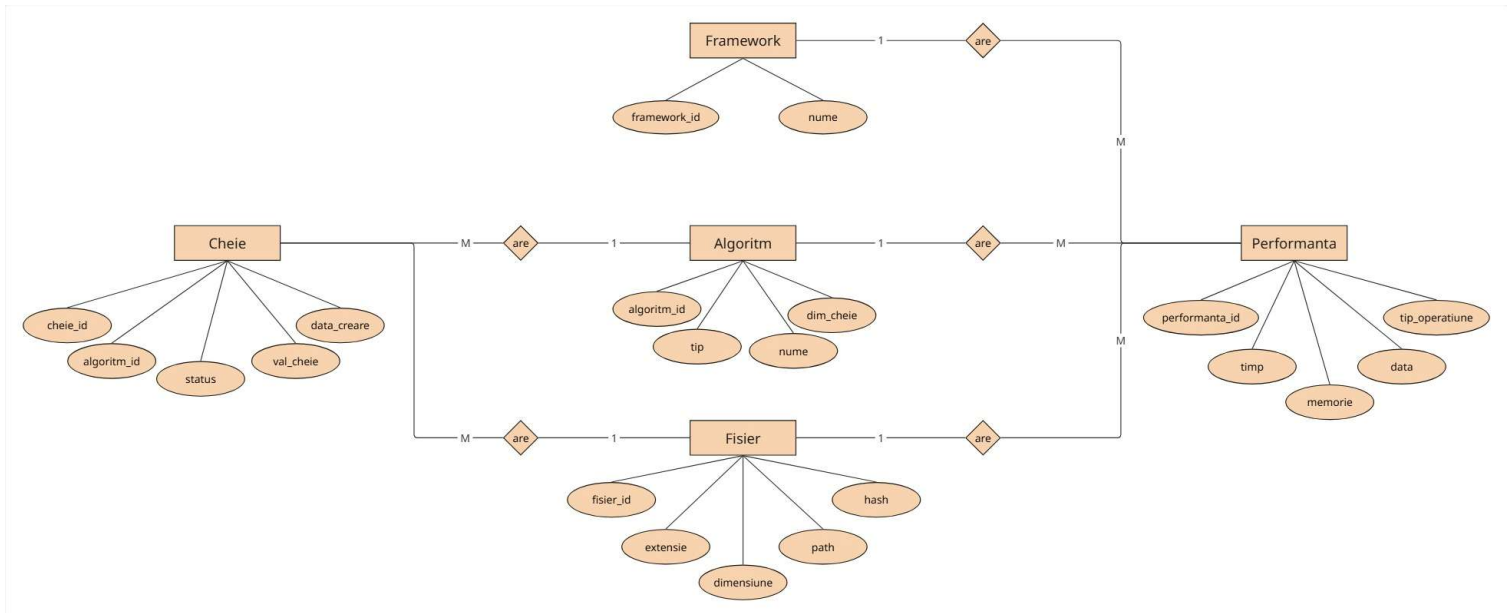
Sistem local de management al cheilor de criptare



Studenți: Vasilca Rareș-Mihai

Simiuc Alexandru

1. Schema proiectului: diagrama entitate-relație (ERD)



Algoritm & Framework: sunt tabele de configurare care stochează detaliile algoritmilor suportați (ex: nume, dimensiune cheie pentru AES/RSA) și motoarele de execuție utilizate (ex: OpenSSL, PyCryptodome).

Fisier: gestionează metadatele fișierelor introduse în sistem (cale, extensie, dimensiune) și amprenta SHA-256 (hash-ul) necesară pentru validarea integrității la decriptare.

Cheie: stochează materialul criptografic generat pentru fiecare fișier (valoarea brută a cheii) și face legătura (Foreign Key) cu algoritmul folosit.

Performanta: funcționează ca un jurnal detaliat (log) de sistem. Înregistrează fiecare acțiune (ex: "Criptare AES"), măsurând timpul de execuție și memoria consumată. Această entitate leagă fișierul prelucrat de algoritmul și framework-ul corespunzător.

2. Algoritmii aleși

Sistemul nostru KMS (Key Management System) utilizează două abordări criptografice fundamentale pentru a demonstra flexibilitatea securizării datelor:

A. Criptarea Simetrică: AES-256-CBC (Advanced Encryption Standard)

De ce l-am ales: fiind un algoritm simetric, folosește aceeași cheie de 256 de biți (32 bytes) atât pentru criptare, oferind atât securitate, cât și performanță.

Detalii implementare: deoarece CBC necesită ca datele să fie un multiplu al dimensiunii blocului (16 bytes), am integrat un mecanism de *Padding PKCS7*

B. Criptarea Asimetrică: RSA-2048 (Rivest–Shamir–Adleman)

De ce l-am ales: permite securizarea datelor folosind o pereche de chei matematice legate între ele (Cheie Publică pentru criptare și Cheie Privată pentru decriptare).

Detalii implementare: am generat chei de 2048 de biți. Din cauza naturii matematice a RSA, algoritmul nu poate cripta fișiere mari direct. Astfel, am implementat la nivel de backend o limitare hardware de siguranță (maxim 190 bytes per fișier).

3. Detalii de implementare (Porțiuni de cod semnificative)

Arhitectura aplicației se bazează pe decuplarea logică a framework-urilor. Acest lucru s-a realizat expunând rute API (FastAPI) distincte.

Exemplu 1: Protecția stării și criptarea AES folosind PyCryptodome

În această porțiune de cod se observă verificarea stării fișierului pentru prevenirea coruperii prin dublă-criptare, generarea cheilor, aplicarea padding-ului nativ și salvarea metricilor de performanță în baza de date.

```
1 @router.post("/operatii/criptare-aes-pycryptodome")
2 def criptare_aes_pycryptodome(fisier_id: int, db: Session = Depends(get_db)):
3     fisier_db = db.query(Fisier).filter(Fisier.fisier_id == fisier_id).first()
4
5     if fisier_db.cheie_utilizata_id is not None:
6         raise HTTPException(status_code=400, detail="Fisierul este deja
criptat!")
7
8     cheie_bytes = get_random_bytes(32)
9     iv = get_random_bytes(16)
10
11     cheie_db = Cheie(algorithm_id=algorithm_aes.algorithm_id,
val_cheie=cheie_bytes.hex())
12     db.add(cheie_db); db.commit()
13
14     with open(fisier_db.path, 'rb') as f:
15         date_clare = f.read()
16
17     start_time = time.perf_counter()
18     cipher = PyCryptoAES.new(cheie_bytes, PyCryptoAES.MODE_CBC, iv)
19     date_criptate = cipher.encrypt(pad(date_clare, PyCryptoAES.block_size))
20     durata = time.perf_counter() - start_time
21
```

Exemplu 2: Interfața asincronă și rutarea dinamică

La nivel de frontend s-a implementat o rutare dinamică. În funcție de selecția utilizatorului (Dropdown-uri), interfața construiește ruta API corespunzătoare, demonstrând flexibilitatea multi-framework a sistemului.

```
1
2 let ruta = "";
3 if(actiune === 'criptare') {
4     if(framework === 'openssl' && algoritm === 'aes') ruta =
      '/operatii/criptare-aes';
5     else if(framework === 'pycryptodome' && algoritm === 'rsa') ruta =
      '/operatii/criptare-rsa-pycryptodome';
6
7 }
8 const response = await fetch(`${ruta}?fisier_id=${fisierSelectatID}`, { method:
  'POST' });
```

4. Dificultăți întâmpinate și soluții

Pe parcursul dezvoltării, am întâmpinat o serie de provocări tehnice, în special din cauza operațiunilor criptografice și a containerizării Docker:

Gestionarea padding-ului între framework-uri:

- Dificultate: biblioteca cryptography folosește o sintaxă diferită pentru a adăuga biții de padding (PKCS7) comparativ cu funcția pad() din PyCryptodome. Au apărut erori la decriptare de tip Integritate Eșuată.
- Soluție: am separat clar conductele (pipelines) de date. Fișierele criptate cu un framework sunt marcate distinctiv prin extensii temporare (.enc vs .pycrypto_enc), astfel încât logica de decriptare să aplice exact operațiunea inversă (unpad) specifică motorului matematic care a produs textul cifrat.

Gestionarea pachetelor multipartite pentru upload:

- Dificultate: implementarea funcționalității "Drag & Drop" în interfață a cauzat inițial erori de tip Internal Server Error (500). Serverul FastAPI nu putea decodifica pachetele binare primite prin rețea.
- Soluție: am identificat necesitatea pachetului python-multipart pentru parsarea obiectelor de tip FormData. După adăugarea acestuia în requirements.txt și reconstrucția imaginii Docker, streaming-ul binar către server a fost rezolvat.

5. User Experience

Sistem Management Chei de Criptare

Fișiere în Sistem

Trage un fișier aici sau dă click pentru a încărca

Actualizează Lista

ID: 5 | Nume: rares.bt

Șterge

Path: files/rares.bt.pyrsa_enc

Operațiuni Criptografice

Fișier Țintă Selectat:
Niciun fișier selectat

Alege Framework-ul:
OpenSSL (implicit)

Alege Algoritmul:
AES-256-CBC (Simetric)

Criptează Fișierul

Decriptează Fișierul

Gestiune Chei (Debug API)

Extrage Chei din DB

ID Cheie: 11 | Algoritm: 2

▶ Valoare cheie

Jurnal Sistem

Sistem inițializat. Gata de utilizare.

Interfața aplicației este împărțită în patru module funcționale principale:

- **1. Fișiere în sistem:** permite utilizatorului să încarce fișiere noi (prin Drag & Drop), să vizualizeze lista completă a acestora, să selecteze un fișier țintă printr-un simplu click sau să șteargă fișierele nedorite.
- **2. Operațiuni criptografice:** permite configurarea procesului de securizare prin alegerea motorului (OpenSSL sau PyCryptodome) și a algoritmului (AES sau RSA). De aici se declanșează efectiv criptarea sau decriptarea fișierului selectat anterior.
- **3. Gestiune chei (Debug API):** modul dedicat monitorizării și testării. Permite extragerea cheilor generate din baza de date, vizualizarea valorilor lor brute (în format Hex sau PEM) și ștergerea acestora pentru a simula pierderea cheii.
- **4. Jurnal sistem:** o consolă integrată care oferă feedback în timp real, afișând statusul operațiunilor (succes), erorile de validare sau mesajele de sistem.
- **5. Analiză performanță vizuală:** am integrat un modul bazat pe Chart.js care afișează dinamic ultimele 10 operațiuni. Acest grafic oferă o comparație vizuală imediată a timpilor de execuție între diferiți algoritmi și framework-uri, oferind un feedback vizual fluid, integrat în tematica aplicației.

